🤖 AI Resume Screening System

An intelligent system to analyze and rank resumes based on job descriptions using NLP techniques

---

📌 Description

This project leverages Natural Language Processing (NLP) to automatically analyze and rank resumes according to a given job description. It helps in identifying the most suitable candidates by comparing resume content with job requirements.

🔧 Technologies Used

- Python
- NLTK
- Scikit-learn
- TF-IDF Vectorization
- Cosine Similarity

🎯 Goal

To automate the resume screening process and assist in selecting the most relevant candidate efficiently and accurately.

1] Import Required Libraries.

In [1]:
```
import PyPDF2        # reads PDF
import nltk          # text cleaning

import os
import sys
sys.path.append(os.path.abspath('.'))

from sklearn.feature_extraction.text import TfidfVectorizer         # converts text
from sklearn.metrics.pairwise import cosine_similarity              # compares text

import re
```

```
nltk.download('stopwords')
from nltk.corpus import stopwords
```

```
[nltk_data] Downloading package stopwords to C:\Users\Apeksha
[nltk_data]     Muluk\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

2] Extract Text from PDF.

In [2]:
```
import PyPDF2

def extract_text(file_path):
    text = ""

    with open(file_path, 'rb') as file:
        reader = PyPDF2.PdfReader(file)

        for page in reader.pages:
            try:
                text += page.extract_text()
            except:
                pass

    return text
```

3] Preprocess Text

In [3]:
```
import re
from nltk.corpus import stopwords

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z ]', '', text)

    words = text.split()
    stop_words = set(stopwords.words('english'))

    words = [word for word in words if word not in stop_words]

    return " ".join(words)
```

4] Convert text to Numbers

In [4]:
```
from sklearn.feature_extraction.text import TfidfVectorizer

def vectorize(texts):
    vectorizer = TfidfVectorizer(
        ngram_range=(1,2),
        stop_words='english'
    )
    vectors = vectorizer.fit_transform(texts)
    return vectors.toarray()
```

5] Similarity Calculation

In [5]:
```
from sklearn.metrics.pairwise import cosine_similarity

def compute_similarity(resume_vectors, job_vector):
    scores = cosine_similarity(resume_vectors, job_vector.reshape(1, -1))
    scores = scores.flatten()

    # 🔥 Optional Boost
    scores = scores * 1.5

    return scores
```

5] Ranking Resumes

In [6]:
```
def rank_resumes(scores):
    ranked = sorted(
        list(enumerate(scores)),   # [(index, score)]
        key=lambda x: x[1],        # sort by score
        reverse=True               # highest first
    )
    return ranked
```

6] Import Modules And Set Path

In [7]:
```
from src.parser import extract_text
from src.preprocess import preprocess
from src.vectorizer import vectorize
from src.matcher import compute_similarity
from src.ranker import rank_resumes

# MAIN FUNCTION: ANALYZE AND RANK RESUMES

def analyze_resumes(uploaded_files, job_desc):

    resumes = []

    for file in uploaded_files:
        text = extract_text(file)
        clean_text = preprocess(text)
        resumes.append(clean_text)

    job_desc_clean = preprocess(job_desc)

    all_text = resumes + [job_desc_clean]

    vectors = vectorize(all_text)

    resume_vectors = vectors[:-1]
    job_vector = vectors[-1]

    scores = compute_similarity(resume_vectors, job_vector)
```

```
    result = rank_resumes(scores)

    return result
```

7] Input Data (Resumes + Job Description)

In [8]:
```
uploaded_files = [
    r"C:\Users\Apeksha Muluk\Downloads\Suraj_Jawale.pdf",
    r"C:\Users\Apeksha Muluk\Downloads\Tejal_Padwal_.pdf",
    r"C:\Users\Apeksha Muluk\Downloads\Apeksha_Muluk_Resume.pdf",
    r"C:\Users\Apeksha Muluk\Downloads\Rohan_Gaikwad.pdf",
    r"C:\Users\Apeksha Muluk\Downloads\Sushil Ambekar resume.pdf",
    r"C:\Users\Apeksha Muluk\Downloads\VAISHNAVI DUMBRE_RESUME.pdf",
    r"C:\Users\Apeksha Muluk\Downloads\RutujaMunde_resume.pdf",
    r"C:\Users\Apeksha Muluk\Downloads\ghodake_siddhi.pdf"
]

job_desc = """Python SQL Excel Power BI Data Analysis Data Cleaning Data Visualizat
            Scikit-learn Dashboard Data Analysis Data Cleaning Data Visualization P
            Machine Learning Statistics Dashboard Data Analysis Python SQL Excel Po
            Learning Pandas NumPy Matplotlib Scikit-learn"""
```

8] Output: Display Ranked Results

In [9]:
```
result = analyze_resumes(uploaded_files, job_desc)

for rank, (index, score) in enumerate(result):
    name = os.path.basename(uploaded_files[index])

    percentage = score * 100
    status = "Selected ✅" if score > 0.25 else "Rejected ❌"

    print(f"Rank {rank+1}: {name}")
    print(f"Score: {score:.2f} ({percentage:.1f}%) → {status}")
    print("-" * 40)

# Top candidate
top_index, top_score = result[0]
top_name = os.path.basename(uploaded_files[top_index])

print(f"\n🏆 Best Candidate: {top_name} ({top_score:.2f})")
```

```
Rank 1: Suraj_Jawale.pdf
Score: 0.38 (38.4%) → Selected ✅
----------------------------------------
Rank 2: Apeksha_Muluk_Resume.pdf
Score: 0.33 (32.5%) → Selected ✅
----------------------------------------
Rank 3: Sushil Ambekar resume.pdf
Score: 0.23 (23.0%) → Rejected ❌
----------------------------------------
Rank 4: VAISHNAVI DUMBRE_RESUME.pdf
Score: 0.21 (21.3%) → Rejected ❌
----------------------------------------
Rank 5: ghodake_siddhi.pdf
Score: 0.12 (11.6%) → Rejected ❌
----------------------------------------
Rank 6: Tejal_Padwal_.pdf
Score: 0.11 (10.8%) → Rejected ❌
----------------------------------------
Rank 7: RutujaMunde_resume.pdf
Score: 0.09 (8.7%) → Rejected ❌
----------------------------------------
Rank 8: Rohan_Gaikwad.pdf
Score: 0.00 (0.0%) → Rejected ❌
----------------------------------------

🏆 Best Candidate: Suraj_Jawale.pdf (0.38)
```

📊 Output Explanation

- Each resume is assigned a similarity score based on how closely it matches the job description.
- Higher score indicates better match.
- The system ranks resumes from highest to lowest score.
- A threshold is used to classify candidates as Selected or Rejected.
- The top-ranked resume is highlighted as the Best Candidate.

In [ ]: